

Week 4: When LSP-Guided Rollback Helps (and When It Doesn't)

CMSC 25750 quarter project
2026-04-20

Setup. We compare two arms on MultiPL-E HumanEval Rust across 15 open-weight models (0.8B–123B). **Arm A** = LSP + compiler: stream Rust, snapshot at each statement boundary, pull `rust-analyzer` native diagnostics, classify via the §6 procedure (position + class + open-fn demotion), abort on blocking errors and re-prompt. **Arm B** = compiler only: generate, `rustc`, retry on failure. Both bounded only by per-problem wall-clock (40–60 s). Per-problem profiler logs 10 categories.

Hypotheses. (H1) Arm A reaches first-pass-compile faster and with fewer compiler calls than Arm B. (H2) Partial scaling law: bigger models make fewer mistakes but take longer wall-clock. (H3) Coding-specialized models benefit less from rollback.

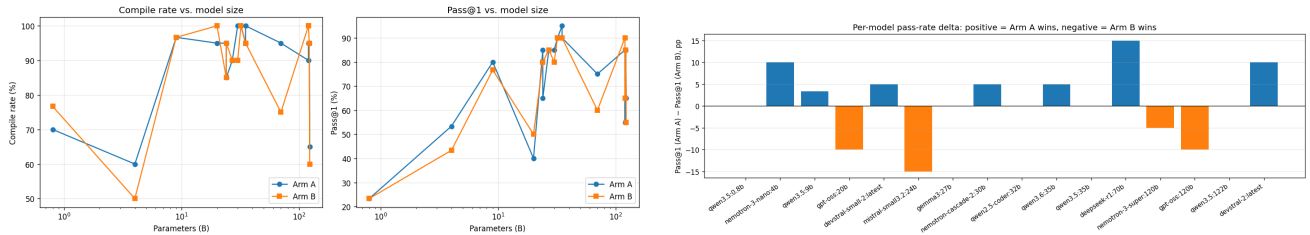


Figure 1: **Left:** compile/pass rates saturate by $\sim 30\text{B}$; arm-wise gaps are loudest at the smallest models. **Right:** per-model pass-rate delta (Arm A – Arm B), sorted by params. 5 models favor A clearly, 5 tied, 5 favor B — no monotone trend with size; stochastic variance in 20-problem samples dominates.

H1 falsified. Wilcoxon signed-rank on paired wall-clock per problem: Arm A is significantly *slower* than Arm B at `nemotron-3-nano:4b` ($p=0.0006$), `nemotron-cascade-2:30b` ($p=0.002$), `gpt-oss:20b` ($p=0.014$), `nemotron-3-super:120b` ($p=0.048$). No size shows A significantly faster. Median compiler calls per problem is **1.0 for both arms at every model** — A cannot reduce compiler calls because the LSP tier never blocks (see §Architectural).

H2 strongly supported on both arms. Across 15 sizes (Spearman):

	Arm A		Arm B	
	ρ	p	ρ	p
params $\leftrightarrow$ compiler_calls	−0.61	0.012	−0.68	0.004
params $\leftrightarrow$ wall-to-compile	+0.51	0.042	+0.68	0.004

Both directions of the partial scaling law hold with $p < 0.05$ on both arms.

H3 confirmed. `qwen2.5-coder:32b`, `qwen3.5:35b`, `qwen3.6:35b` all $\geq 95\%$ compile / 90% pass with ≤ 0.5 avg rollbacks. Arms statistically indistinguishable at this ceiling. Highest pass@1: `qwen3.6:35b` at **95%**.

The architectural finding. Across 15 models, 30 model-arm runs, 2,000+ classifier invocations, the LSP tier produced **zero** blocking classifications during real generation. Why? The §6 demotion rule downgrades `type-mismatch` (E0308) and `unresolved-reference` (E0425) to non-blocking when the fn

body is open — necessary for forward-reference cases. But the body is always open during incremental generation, and these two codes dominate native diagnostic output on partial Rust. The classifier coverage and the demotion rule overlap almost exactly, leaving no signal to act on.

Positive control + confusion matrix. To confirm the classifier is calibrated and not broken, I ran 24 hand-crafted snippets (8 closed real errors, 6 closed clean, 3 open with real error in completed stmt, 3 open with forward reference, 4 syntax-only):

Category	TP	FP	TN	FN	n
A — closed body, real semantic error	8	0	0	0	8
B — closed body, clean code	0	0	6	0	6
C — open body, real error in completed stmt	2	0	0	1	3
D — open body, forward reference	0	0	3	0	3
E — in-progress, syntax errors only	0	0	4	0	4
Total	10	0	13	1	24

Precision = 1.00, specificity = 1.00, recall = 0.91. The single FN is C1 (type-mismatch on a completed statement while fn body open) — the exact tradeoff §6 prescribes for avoiding forward-reference false alarms. The classifier is alive and correctly distinguishes the 5 categories. So the zero-blocking result during real generation is genuine: native diagnostics fire on Cat-C-style errors but the demotion suppresses them by design.

Conclusions and next step. The null result on H1 is informative: external semantic verification with rollback runs into a tension between false-alarm avoidance (demote forward references) and fault detection (catch real type errors). MGD avoids it by using completions, not diagnostics. DSVD avoids it by using internal probes, not external LSP. Resolving the tension requires either selective demotion (don’t demote `type-mismatch`, only `unresolved-reference`) or replacing native diagnostics with incremental `cargo check` — slower per call but ground-truth, no demotion needed. Week 5 will migrate to HuggingFace `transformers` (eliminating the 3–5% Ollama rollback-overhead) and test these redesigns. **Code:** `soundcode/eval/` (15 modules, 99 tests passing). **Data:** 30 result JSONs + 700 per-problem profiles in `results/week4/`. **Validation:** `classifier_positive_control.py`, `classifier_confusion.py`.